

Netflix User Behavior & Anime Recommendation Engine

An End-to-End Data Engineering Pipeline with SQL Optimization

Prepared by: Team

Project Overview & Core Objectives

- **What This Project Does:** Implements an end-to-end data pipeline analyzing user viewing trends and offering targeted cross-platform recommendations.
- **Core Value:** Bridges raw business data metrics with real analytical strategies to lower churn rates and boost session length.
- **Ecosystem Integration:** Merges production streaming profiles with community-verified entertainment databases to build automated lookahead funnels.
- **User Engagement Tracking:** Segmenting metrics by age and country structures to see where engagement peaks.
- **Binge Behavior Breakdown:** Isolating triggers that prolong sessions to pinpoint high-retention titles.
- **Proactive Churn Mitigation:** Correlating interactions and login delays with exit loops to protect lifetime value.
- **Personalized Anime Mapping:** Matching viewing categories with global ratings to serve custom recommendations.

Data Architecture & Acquisition Sources

- **Netflix User Behavior Dataset:** Demographics (age, country), engagement metrics (watch time, binge sessions, completion rates), and account data (subscription type, churn status).
- **Netflix Titles Catalog:** Metadata for the Netflix library, including titles, directors, countries, and genre classifications.
- **Anime Database:** External anime metadata, including titles, genres, and community-verified ratings.

Source Schema Manifest: Dataset Columns Breakdown

- **1. netflix_user_behavior_dataset.csv:** user_id, age, gender, country, account_age_months, subscription_type, monthly_fee, payment_method, primary_device, devices_used, favorite_genre, avg_watch_time_minutes, watch_sessions_per_week, binge_watch_sessions, completion_rate, rating_given, content_interactions, recommendation_click_rate, days_since_last_login, churned
- **2. netflix_titles.csv:** show_id, type, title, director, cast, country, date_added, release_year, rating, duration, listed_in, description
- **3. anime.csv:** anime_id, name, genre, type, episodes, rating, members

ETL Pipeline Architecture & Engineering Blueprint

- **Data Extraction Module (`extract.py`)**: Automates raw CSV data ingest paths cleanly across relative system environments to fetch Netflix and anime properties.
- **Data Transformation Module (`transform.py`)**: Performs continuous numeric semantic binning via Pandas to compile categorical demographic age groups.
- **Data Loading Backend Module (`load.py`)**: Establishes optimized relational transaction connections to an isolated local SQLite storage layer.

Pipeline Extraction Script: extract.py Code Review

```
import pandas as pd

def extract_netflix_data():
    df = pd.read_csv(r"C:\Users\Admin\Desktop\anime_sql_project\data\netflix_titles.csv")
    return df

def extract_anime_data():
    df = pd.read_csv(r"C:\Users\Admin\Desktop\anime_sql_project\data\anime.csv")
    return df

def extract_rating_data():
    df = pd.read_csv(r"C:\Users\Admin\Desktop\anime_sql_project\data\netflix_user_behavior_dataset.csv")
    return df
```

Pipeline Transformation Script: transform.py Code Review

```
import pandas as pd

def transform_data(anime_df, titles_df, users_df):
    # 1. Add Age Group column using custom bin limits
    bins = [18, 25, 40, 100]
    labels = ['18-25', '26-40', '40+']
    users_df['age_group'] = pd.cut(users_df['age'], bins=bins, labels=labels, include_lowest=True)

    # 2. Add Age Bracket dimension for specialized cohort metrics
    bins_c = [0, 25, 50, 100]
    labels_c = ['Young', 'Adult', 'Senior']
    users_df['age_bracket'] = pd.cut(users_df['age'], bins=bins_c, labels=labels_c, right=False)

    return anime_df, titles_df, users_df
```

Pipeline Load Script: load.py Code Review

```
import sqlite3

def load_data(anime_df, netflix_titles_df, netflix_users_df):
    # Connect to database file backend
    conn = sqlite3.connect('netflix_analysis.db')

    # Commit structures inside relational storage layers
    anime_df.to_sql('anime', conn, if_exists='replace', index=False)
    netflix_titles_df.to_sql('netflix_titles', conn, if_exists='replace', index=False)
    netflix_users_df.to_sql('netflix_users', conn, if_exists='replace', index=False)

    conn.close()
```


Problem 1: Top 3 Favorite Genres per Age Group

```
SELECT age_group, favorite_genre, count
FROM (
  SELECT
    CASE
      WHEN age BETWEEN 18 AND 25 THEN '18-25'
      WHEN age BETWEEN 26 AND 40 THEN '26-40'
      ELSE '40+'
    END AS age_group,
    favorite_genre,
    COUNT(*) as count,
    ROW_NUMBER() OVER(PARTITION BY (CASE WHEN age BETWEEN 18 AND 25 THEN
'18-25' WHEN age BETWEEN 26 AND 40 THEN '26-40' ELSE '40+' END) ORDER BY
COUNT(*) DESC) as rank
  FROM netflix_users
  GROUP BY age_group, favorite_genre
)
WHERE rank <= 3;
```

age_group	favorite_genre	count
18-25	Documentary	1106
18-25	Horror	1056
18-25	Romance	1054
26-40	Sci-Fi	2061
26-40	Romance	2039
26-40	Documentary	2029
40+	Thriller	3225
40+	Documentary	3217
40+	Action	3202

Analytical Strategy: Window Function (ROW_NUMBER OVER PARTITION BY), Nested Subquery, CASE WHEN evaluation | Database Constraints: Primary Key: user_id group stratification mappings

Problem 2: Average Netflix Watch Time per Country

```
SELECT
    country,
    AVG(avg_watch_time_minutes) as avg_netflix_using_time
FROM netflix_users
GROUP BY country;
```

country	avg_netflix_using_time
Australia	152.38
Brazil	154.09
Canada	155.17
France	155.63
Germany	156.19
India	154.56
Japan	153.63
Spain	156.71
UK	155.33
USA	155.77

Problem 3: Genre Dominance & Country Market Coverage

```
SELECT
    favorite_genre,
    GROUP_CONCAT(country, ', ') as countries_where_it_is_the_most_popular
FROM (
    SELECT country, favorite_genre, COUNT(*) as count
    FROM netflix_users
    GROUP BY country, favorite_genre
    HAVING count = (
        SELECT MAX(cnt)
        FROM (SELECT country, favorite_genre, COUNT(*) as cnt FROM netflix_users
              GROUP BY country, favorite_genre) as sub
        WHERE sub.country = netflix_users.country
    )
)
GROUP BY favorite_genre;
```

favorite_genre	countries_where_it_is_the_most_popular
Comedy	Brazil
Documentary	Spain
Drama	India, USA
Horror	France, UK
Romance	Australia, UK
Sci-Fi	Canada, Japan
Thriller	Germany

Analytical Strategy: Nested Subqueries, String Concatenation Aggregation (GROUP_CONCAT), MAX filtering, HAVING clause | Database Constraints: Composite field domain constraints

Problem 4: Content Genres Driving Highest Binge Behavior

```
SELECT
    favorite_genre,
    AVG(binge_watch_sessions) as avg_binge_sessions
FROM netflix_users
GROUP BY favorite_genre
ORDER BY avg_binge_sessions DESC;
```

favorite_genre	avg_binge_sessions
Drama	7.06
Documentary	7.04
Comedy	7.04
Thriller	7.03
Sci-Fi	7.02
Romance	6.96
Horror	6.94
Action	6.94

Analytical Strategy: Aggregation Function (AVG), GROUP BY clause, ORDER BY DESC sorting sequence
| Database Constraints: Continuous numeric behavioral attribute sets

Problem 5: Subscription Tier vs. Average Watch Time

```
SELECT
    subscription_type,
    AVG(avg_watch_time_minutes) as avg_watch_time
FROM netflix_users
GROUP BY subscription_type;
```

subscription_type	avg_watch_time
Basic	155.22
Premium	154.60
Standard	155.00

Problem 6: Movie Volume Distributions per Release Year

```
SELECT
  release_year,
  COUNT(*) AS movies_produced
FROM
  netflix_titles
WHERE
  type = 'Movie'
GROUP BY
  release_year
ORDER BY
  release_year DESC;
```

release_year	movies_produced
2021	277
2020	517
2019	633
2018	767
2017	767
2016	658
2015	398
2014	264
2013	225
2012	173
2011	145

Analytical Strategy: Count Aggregation Function (COUNT), Static WHERE criteria filtering, GROUP BY, ORDER BY DESC sequence
| Database Constraints: Release year dimension index tracking

Analysis A: Engagement Metric Analytics vs. User Churn

```
SELECT
    churned,
    AVG(avg_watch_time_minutes) as avg_watch,
    AVG(binge_watch_sessions) as avg_binge
FROM netflix_users
GROUP BY churned;
```

churned	avg_watch	avg_binge
No	155.16	7.01
Yes	154.10	6.97

Analysis B: Library Metadata Distributions (Netflix)

```
SELECT
  listed_in,
  COUNT(*) as title_count
FROM netflix_titles
GROUP BY listed_in
ORDER BY title_count DESC;
```

listed_in	title_count
Dramas, International Movies	362
Documentaries	359
Stand-Up Comedy	334
Comedies, Dramas, International Movies	274
Dramas, Independent Movies, International Movies	252
Kids' TV	220
Children & Family Movies	215
Children & Family Movies, Comedies	201
Documentaries, International Movies	186
Dramas, International Movies, Romantic Movies	180
Comedies, International Movies	176

Analytical Strategy: Count Aggregation Function (COUNT), GROUP BY grouping, ORDER BY DESC sequencing
| Database Constraints: Listed-in category field dimension constraints

Analysis C: Device Profile Completion Rate Efficiencies

```
SELECT
    primary_device,
    AVG(completion_rate) as avg_completion
FROM netflix_users
GROUP BY primary_device;
```

primary_device	avg_completion
Laptop	64.77
Mobile	64.73
Smart TV	64.22
Tablet	64.41

Cross-Platform Strategy: Anime Recommendations for First 100 Users

```
SELECT
  u.user_id,
  u.favorite_genre,
  a.name AS recommended_anime,
  a.rating
FROM (
  SELECT user_id, favorite_genre
  FROM netflix_users
  LIMIT 100
) AS u
JOIN anime a ON a.genre LIKE '%' || u.favorite_genre || '%'
GROUP BY u.user_id
ORDER BY a.rating DESC;
```

user_id	favorite_genre	recommended_anime	rating
U100099	Romance	Kimi no Na wa.	9.37
U100095	Drama	Kimi no Na wa.	9.37
U100088	Romance	Kimi no Na wa.	9.37
U100080	Romance	Kimi no Na wa.	9.37
U100076	Romance	Kimi no Na wa.	9.37
U100072	Drama	Kimi no Na wa.	9.37
U100069	Romance	Kimi no Na wa.	9.37
U100068	Drama	Kimi no Na wa.	9.37
U100066	Romance	Kimi no Na wa.	9.37
U100061	Romance	Kimi no Na wa.	9.37
U100060	Drama	Kimi no Na wa.	9.37

Analytical Strategy: Subquery limits (LIMIT 100), Wildcard String JOIN matching (LIKE '%...%'), GROUP BY user space aggregation, ORDER BY DESC rating values | Database Constraints: Foreign Key: genre cross-database text alignment constraints

Strategic Insights & Pipeline Value

- Optimized SQL Queries: Window functions (ROW_NUMBER) and nested metrics filter complex layers quickly without overhead.
- Personalized Architecture: Linking genre-specific user metrics to high-performing content maps out precise product cross-selling avenues.
- Actionable Infrastructure: Clean data segmentation separates user patterns by country and subscription level to optimize platform resources.